RESEARCH-ARTICLE

Devirtualization in LLVM

PIOTR PADLEWSKI, University of Warsaw, Warsaw, MA, Poland

Published: 22 October 2017

SPLASH '17: Conference on Systems, Programming, Languages, and Applications: Software for Humanity
October 22 - 27, 2017
BC, Vancouver, Canada

Conference Sponsors:
SIGPLAN

SPLASH Companion 2017: Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity
(October 2017)
https://doi.org/10.1145/3135932.3135947
ISBN: 9781450355148

Devirtualization in LLVM

Piotr Padlewski

University of Warsaw, Poland

piotr.padlewski@gmail.com

Abstract

Devirtualization is an optimization changing indirect (virtual) calls to direct calls. It improves performance by allowing extra inlining and removal of redundant loads. This paper presents a novel way of handling C++ devirtualization in LLVM by unifying virtual table loads across calls using different SSA values to represent different dynamic types.

CCS Concepts • **Software and its engineering** → *Source code generation*; Polymorphism;

Keywords devirtualization, llvm, C++, virtual function, indirect call

ACM Reference Format:

Piotr Padlewski. 2017. Devirtualization in LLVM. In *Proceedings of 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion'17).* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3135932.3135947

1 Introduction

Every virtual function call consists of 3 instructions: a virtual table (vtable) load using a virtual pointer (vptr) stored in the object, a virtual function load from vtable and an actual call.

```
%vtable = load {...} %a
%vfunction = load {...} %vtable
call void %vfunction(%struct.A* %a)
```

The virtual call can have a huge performance cost – the branch predictor often is unable to predict indirect jumps accurately, resulting in processor waiting for vtable load to finish. There are many ways of handling devirtualization [2][3][4][5][6][7]. The basic principle behind devirtualization is the "store to load" propagation – the value of vptr is determined by finding the store instruction that defines it. Unfortunately, in many cases it is difficult to devirtualize a call, for example, if it is preceded by a call to an outlined function that could potentially change the value of vptr:

```
void f(A *a) {
  a->foo(); // virtual call
  clobber(a);
  a->foo();
}
void clobber(A *a) {
  new (a) B;
}
```

Here "clobber" function uses the "placement new" mechanism, which constructs an object of type `B` in the place of `a`, possibly changing the vptr of `a` (if the dynamic type was different from `B`). Fortunately, according to the C++ standard if the dynamic type changed, then the second virtual call would result in an Undefined Behavior. This is because the pointer would point to an object whose lifetime has ended [1]. The only legal use of the object created by placement new is by using the pointer returned by it.

1.1 The Model

To indicate that multiple virtual calls on an object use the same virtual table, we introduced `invariant group` metadata [1]. The presence of this metadata on an instruction tells the optimizer that every load and store to the same pointer can be assumed to load or store the same value. The frontend (Clang) decorates all vtable loads and stores with this metadata.

Invariant group metadata is bound to the SSA (Static Single Assignment) value of pointer operand. This means that pointers to the same memory location, but with different dynamic type must have different SSA values - e.g. the pointer returned from the placement new needs to have different SSA value. In the example below, pointers `a` and `b` could be wrongly optimized to share the same SSA value because the optimizer figures out they are bitwise identical.

```
A *a = new A;
A *b = new (a) B;
```

[1] Instruction metadata are extra information used as a hint for the optimizer and can be stripped anytime preserving semantics.

SPLASH Companion'17, October 22–27, 2017, Vancouver, Canada

ACM ISBN 978-1-4503-5514-8/17/10…$15.00
https://doi.org/10.1145/3135932.3135947

The `i8* @llvm.invariant.group.barrier(i8*)` intrinsic [2] has been introduced for that purpose. It returns another pointer that aliases its argument but which is considered different for the purposes of load/store invariant.group metadata.

There are a few places where a dynamic object might change – when constructing or destructing an object, when placement new is used or when a union has different dynamic objects. The LLVM code for the previous example:

```
%a = call i8* @_new(i64 8)
call void @A_Constructor(%struct.A* %a)
%b = call i8* @llvm.invariant.group.barrier(i8*%a)
call void @B_Constructor(%struct.B* %b)
```

However, an open issue remains how to optimize pointers equality based on comparison without skipping the barriers.

Moreover, to indicate that a vtable never changes, we mark vptr loads with `invariant.load` metadata, which allows LLVM to remove redundant loads. So, for example:

```
a->foo();
a->foo();
```

will be optimized to

```
%vtable = load {...} %a, !invariant.group !{}
%vfun = load {...} %vtable, !invariant.load !{}
call void %vfun(%struct.A* %a)
call void %vfun(%struct.A* %a)
```

1.2 Outlined Constructor

In order to fully devirtualize an indirect call, the value of vptr must be known. The optimizer can figure out the value only by finding the corresponding store instruction, which happens in a constructor. This means that if the constructor is not inlined (because it is too large, or external), no virtual calls on the object will be devirtualized. To not rely on constructor's availability or inlining, we decided to use the @llvm.assume intrinsic to indicate the value of vptr. Assume is akin to assert – optimizer seeing a call to `@llvm.assume(i1 %b)` can assume that `%b` is true after it. We can indicate vptr value by comparing it with the vtable and then call the `@llvm.assume` with the result of this comparison.

```
call void @A_Constructor(%struct.A* %a)
%vtable = load {...} %a
%b = icmp eq %vtable, @_A_vtable
call void @llvm.assume(i1 %b)
```

1.3 Optimizing the Barriers

Introducing the `invariant.group.barrier` intrinsic should not prevent crucial optimizations from happening. The three most important optimizations are DSE (Dead Store Elimination) – because dead vtable stores are very common in constructors and destructors of derived classes, store to load propagation and redundant load elimination – because of member value propagation from the constructors.

The most elegant solution was to teach the AA (Alias Analysis) that the pointer returned from a barrier aliases its argument. DSE required a minor fix and GVN did not require any fixes in order to optimize these cases based on information from AA.

[2] Intristic functions are an extension mechanism for the LLVM language, having well known semantics.

2 Experimental Results

There are 4 numerical benchmarks that author used to evaluate devirtualization:

- Number of vtable loads replaced
- Number of vtable uses devirtualized
- Number of vfunction pointers loads replaced
- Number of virtual function uses devirtualized

Replacing vtable or vfunction load occurs when the optimizer merges two loads referring to the same value, or when a load is replaced with constant. A number of vtable uses or vfunction uses devirtualized specify how many loads got replaced by a constant.

The benchmarks were collected while compiling LLVM and ChakraCore project (a Javascript engine that powers Microsoft Edge) showing a big improvement.

Results for LLVM			
statistic	baseline	devirt	diff
# of vtable loads replaced	1451	14254	882.36%
# of vtable uses devirtualized	982	3269	232.89%
# of vfunction loads replaced	1084	9388	766.05%
# of vfunction devirtualized	954	1861	95.07%
Results for ChakraCore			
statistic	baseline	devirt	diff
# of vtable loads replaced	126	2465	1856.35%
# of vtable uses devirtualized	17	584	3335.29%
# of vfunction loads replaced	45	1082	2304.44%
# of vfunction devirtualized	32	131	309.38%

3 Conclusion

Experiments show that the use of invariant group metadata in the LLVM compiler, together with an artificial barrier intrinsic to prevent unintended vptr identifications, results in a significant improvement in the number of devirtualized function calls. Future work includes hoisting vtable loads out of the loops, speculative devirtualization and fixing an open issue with optimizing pointers equality based on comparison without skipping the barriers.

References

[1] 2017. Working Draft, Standard for Programming Language C++:Objects lifetime. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4659.pdf. (21 March 2017).

[2] Brad Calder and Dirk Grunwald. 1994. Reducing Indirect Function Call Overhead in C++ Programs. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '94)*. ACM, New York, NY, USA, 397–408. https://doi.org/10.1145/174675.177973

[3] Peter Collingbourne. 2017. Devirtualization using LTO with Clang. https://clang.llvm.org/docs/LTOVisibility.html. (16 March 2017).

[4] Jan Hubicka. 2014. Devirtualization in C++. (2014). http://hubicka.blogspot.com/2014/01/devirtualization-in-c-part-1.html

[5] Jose A. Joao, Onur Mutlu, Hyesoon Kim, Rishi Agarwal, and Yale N. Patt. 2008. Improving the Performance of Object-oriented Languages with Dynamic Predication of Indirect Jumps. *SIGOPS Oper. Syst. Rev.* 42, 2 (March 2008), 80–90. https://doi.org/10.1145/1353535.1346293

[6] Hyesoon Kim, José A. Joao, Onur Mutlu, Chang Joo Lee, Yale N. Patt, and Robert Cohn. 2007. VPC Prediction: Reducing the Cost of Indirect Branches via Hardware-based Dynamic Devirtualization. *SIGARCH Comput. Archit. News* 35, 2 (June 2007), 424–435. https://doi.org/10.1145/1273440.1250715

[7] Jason Mccandless and David Gregg. 2012. Compiler Techniques to Improve Dynamic Branch Prediction for Indirect Jump and Call Instructions. *ACM Trans. Archit. Code Optim.* 8, 4, Article 24 (Jan. 2012), 20 pages. https://doi.org/10.1145/2086696.2086703